

DOM manipulation med JS

💡 Dagens fokus

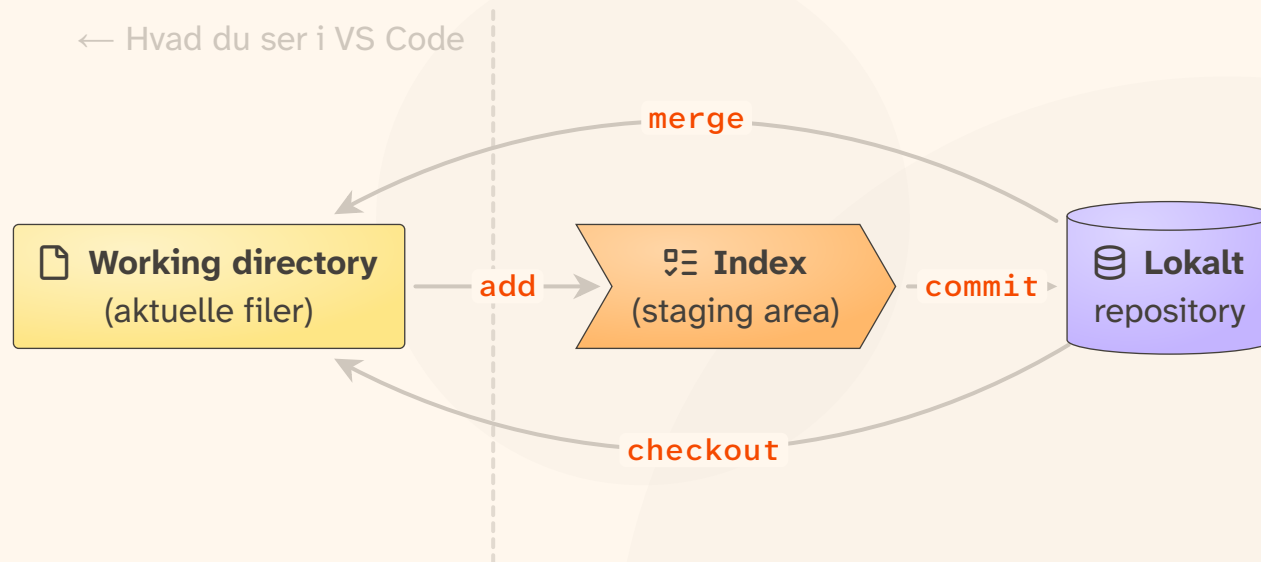
- Git remotes og fetch vs pull
- Hvordan JavaScript loades og eksekveres i browseren
- Hvordan man manipulerer DOM'en og reagerer på brugerens handlinger



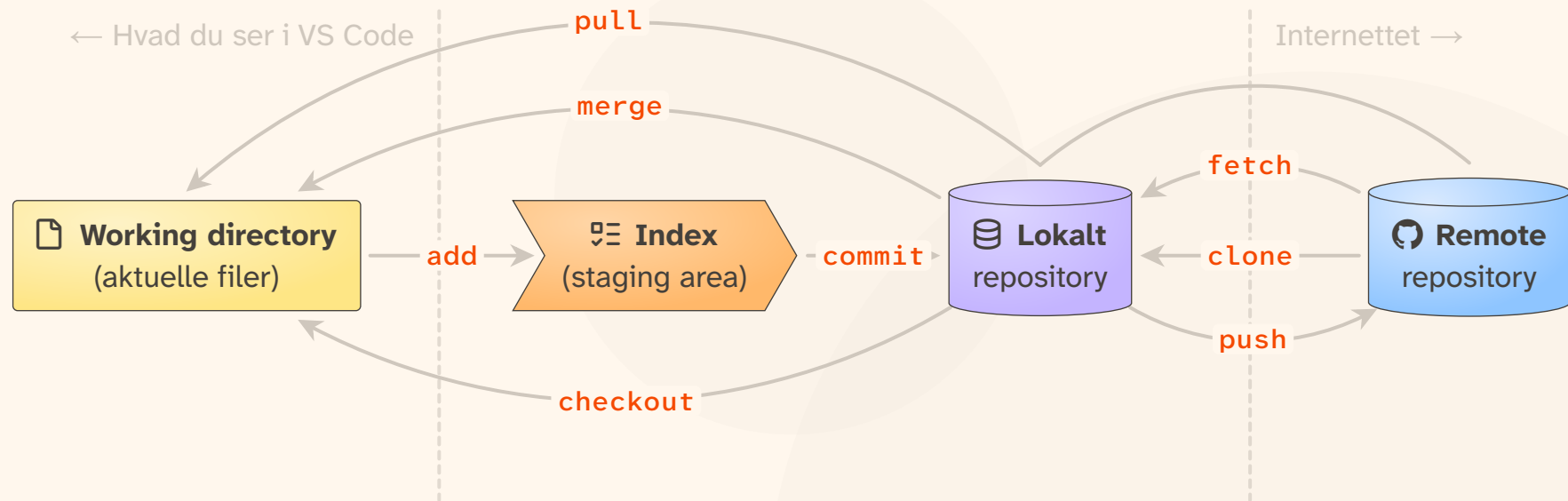
Dagens program

8:15	5	🎓	Intro & agenda
8:20	15	🎓	<u>Intro til Git remotes, push, fetch & merge vs pull</u>
8:35	20	🤝	<u>Leg med remote visualisering</u>
8:55	10	🎓	<u>Intro til DOM: Document Object Model</u>
9:05	15	🎓	<u>Script loading (blocking, async, defer, module)</u>
9:20	20	🤝	<u>Hands-on: Afprøv script loading strategier</u>
9:40	20	🕒	Pause
10:00	10	🎓	<u>Events og addEventListener()</u>
10:10	15	🤝	<u>Afprøv events</u>
10:25	10	🎓	<u>DOM manipulation: innerText og class</u>
10:35	50	🛠️	<u>Byg en character count i din chatbot</u>
11:25	5	🎓	Opsamling

1. Intro til Git remotes, push, fetch & merge vs pull



1.1 Git workflow: remote



1.2 Grundlæggende remote kommandoer

Tilføj et nyt remote repository:

Kommando *arg* Navn på remote

```
git remote add origin https://github.com/user/repo.git
```

Program Subkommando *arg* URL til remote

Se eksisterende remotes:

Kommando

```
git remote -v
```

Program *flag* Verbose

Hent ændringer fra remote:

Kommando

```
git fetch origin
```

Program *arg* Remote navn

Push lokale commits til remote:

Kommando *arg* Branch navn

```
git push origin main
```

Program *arg* Remote navn

Hent + merge ændringer fra remote:

Kommando *arg* Branch navn

```
git pull origin main
```

Program *arg* Remote navn

1.3 Fetch vs Pull: Hvad er forskellen?

📄 Fetch først, merge manuelt

Kommando
`git fetch origin`
Program arg Remote navn

Kommando
`git merge origin/main`
Program arg Remote/Branch

📄 Pull: Fetch + Merge i én kommando

Kommando arg Branch navn
`git pull origin main`
Program arg Remote navn

💡 Fetch: Hent uden at merge

`git fetch` henter ændringer fra remote repository, men merger dem **ikke** automatisk:

- Downloader nye commits, branches og tags
- Opdaterer remote-tracking branches (f.eks. `origin/main`)
- Dit lokale arbejde forbliver uændret
- Giver dig mulighed for at inspicere ændringer før merge

1.4 Leg med remote visualisering

🔗 Parvis ⌚ 20 min

Afprøv en interaktiv visualisering for at se hvordan commits bevæger sig mellem lokale og remote repositories.

🔗 Sammen med din sidemakker:

01 Lav lokale commits og push:

- Åbn git-school.github.io/visualizing-git
- Vælg **Free Explore with Remote** fra dropdown-menuen
- Lav commits med `git commit`
- Observer hvordan `origin` først får besked om dine commits når du pusher
- Prøv også at lave en ny branch med `git branch {branch navn}` og skift til den med `git checkout {branch navn}`
- Lav commits på din branch og observer hvordan din remote først hører om branchen når du pusher den.

02 Skift perspektiv:

- Brug kommandoerne `mode local` og `mode remote` for at skifte mellem det lokale og remote repository
- Prøv at lave commits på dem begge og brug `fetch`, `pull` og `push` for at udveksle dem
- Forstå hvordan de er separate indtil du synkroniserer.

03 Hent ændringer fra en remote:

- Vælg **Upstream Changes** fra dropdown-menuen
- Først: Brug `git fetch` og merge manuelt bagefter
- Skriv `clear` for at nulstille og starte forfra
- Derefter: Brug `git pull` (som er fetch + merge i én kommando).

1.5 Arbejd med flere remotes

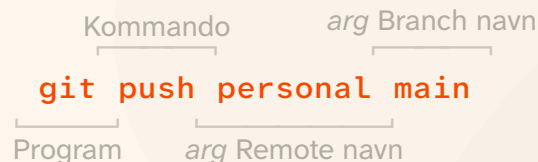
Tilføj en ekstra remote:


`git remote add personal https://github.com/username/repo.git`

Push først til origin:


`git push origin main`

Når du er tilfreds med dit arbejde, så push til dit eget public repo:


`git push personal main`

Typiske remote konfigurationer

Forking workflow (fx. open source projekter):

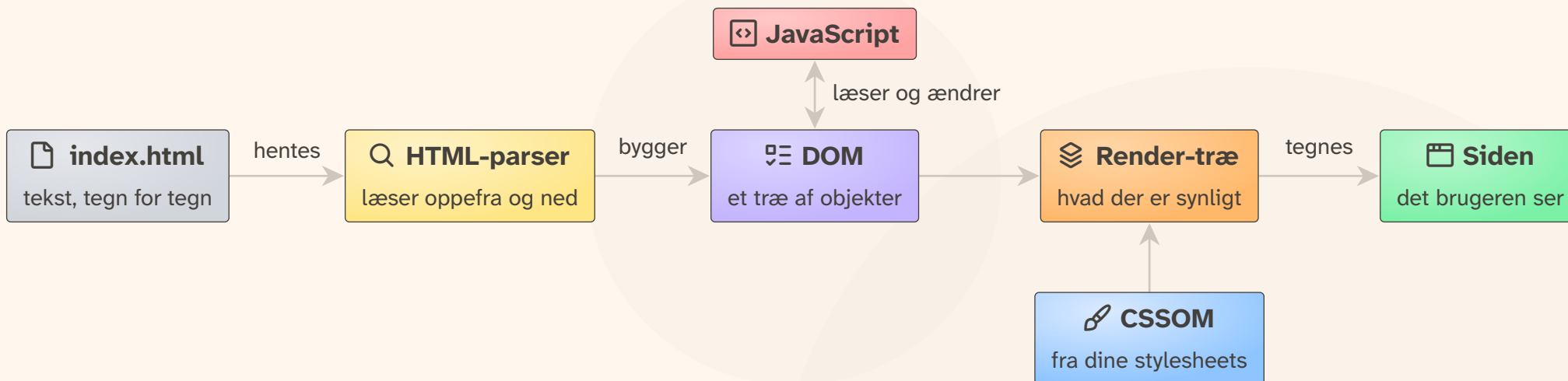
- **origin**: Dit forkede repository
- **upstream**: Det originale repository du forkede fra

Projekter fra studiet:

- **origin**: GitHub Classroom repository
- **personal**: Dit personlige repository til portfolio

2. Intro til DOM: Document Object Model

HTML-filen er **tekst** — opskriften på siden, ikke siden selv. Mens browseren læser HTML'en (oppefra og ned), bygger den et **træ af objekter**. Det træ hedder DOM'en, og det er det, der bliver tegnet i browseren.

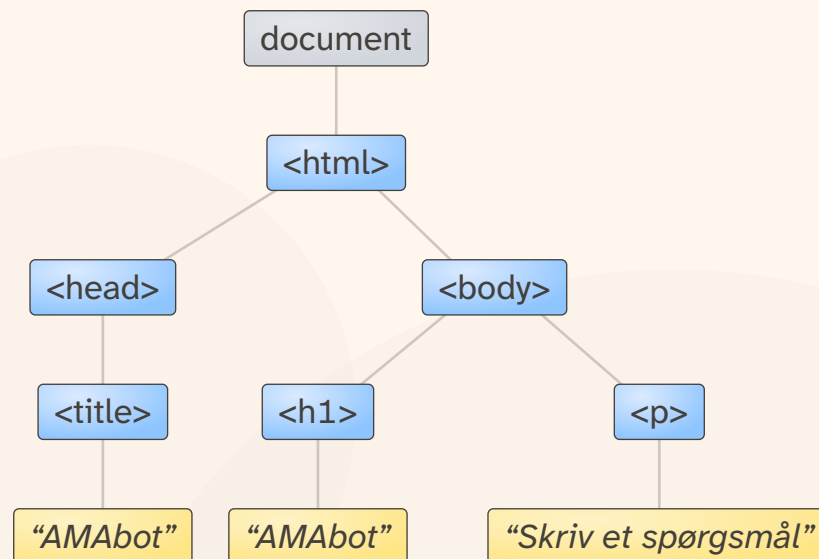


💡 Bemærk:

- JavaScript ændrer aldrig i selve HTML-filen — kun i DOM'en. Og browseren opdaterer sin visning derefter.
- **View source** viser filen fra serveren. DevTools' **Elements**-fane viser DOM'en, som den er lige nu. De er sjældent ens.

2.1 Fra tekst til træ

```
<html>
  <head>
    <title>AMAbot</title>
  </head>
  <body>
    <h1>AMAbot</h1>
    <p>Skriv et spørgsmål</p>
  </body>
</html>
```



Bemærk at **tekst også er noder** (de gule). Derfor er **innerText** en property på elementet omkring teksten, og ikke på teksten selv.

Hvert element bliver til et objekt med properties (**innerText**, **classList**, **value**, **style** ...) og en liste af “child nodes”.

document.querySelector() er jeres indgang til træet, og den bruger helt almindelige CSS selectors.

3. Script loading (blocking, async, defer, module)

Browseren har forskellige måder at load JavaScript på, som påvirker performance og brugeroplevelse betydeligt:

Blocking loading (standard):

- Browser stopper HTML parsing
- Script downloades og køres øjeblikkeligt
- HTML parsing fortsætter først efter script er færdigt
- Kan blokere rendering af siden

```
<script src="script.js"></script>
```

Defer loading:

- Script downloades parallelt med HTML parsing
- Køres først når HTML parsing er færdig
- Scripts køres i den rækkefølge de optræder
- Ideelt til scripts der manipulerer DOM

```
<script src="script.js" defer></script>
```

Async loading:

- Script downloades parallelt med HTML parsing
- Køres så snart det er downloadet (afbryder parsing)
- Rækkefølge af scripts er ikke garanteret
- Bedst til uafhængige scripts

```
<script src="script.js" async></script>
```

Performance påvirkning:

- Blocking scripts kan forsinke First Contentful Paint (FCP)
- Async scripts kan forstyrre brugeroplevelsen hvis de køres for tidligt
- Defer scripts er ofte den sikreste strategi for DOM manipulation
- Kombinationen af strategier giver optimal loading

4. Hands-on: Afprøv script loading strategier

🤝 Parvis ⌚ 20 min

📄 index.html

```
<head>
  <script>
    document.addEventListener("DOMContentLoaded", function () {
      console.log("-- DOMContentLoaded")
    })
    window.addEventListener("load", function () {
      console.log("-- load")
    })
  </script>
  <script src="blocker.js"></script>
</head>
<body>
  <h1 id="top">Kan du se mig?</h1>
  <!-- og mindst en skærmfuld tekst mere -->
</body>
```

📄 blocker.js

```
console.log("start",
Math.round(performance.now()), "ms")
console.log("fandt h1:",
document.querySelector("#top"))

let start = Date.now()
while (Date.now() - start < 3000) {
  // blokerer hovedtråden i 3 sekunder
}

console.log("slut ",
Math.round(performance.now()), "ms")
```

4.1 Fem varianter – kør dem én ad gangen

For hver variant: gem, hard-reload, og notér hvad I ser. Kig både på **siden** og i **konsollen**.

🔧 Sæt DevTools op først

01 Network-fanen → **Throttling: Slow 4G**

02 Sæt flueben i **Disable cache**

03 Hold Console-fanen åben ved siden af

Uden throttling henter localhost det hele på under et millisekund, og så ligner alle fem varianter hinanden.

1/3 Varianterne:

01 `<script src="blocker.js">` i `<head>`

02 Samme tag flyttet ned lige før `</body>`

03 Tilbage i `<head>`, tilføj `async`

04 Skift `async` ud med `defer`

05 Skift `defer` ud med `type="module"`

🔍 Kig efter:

→ **Siden:** er den blank i tre sekunder — eller nåede teksten at blive tegnet, før alt frøs?

→ **Konsollen:** stod der et `<h1>`-element eller `null` efter `fandt h1:?`

→ **Rækkefølgen:** kørte scriptet før eller efter `-- DOMContentLoaded?`

→ **Network:** hvornår starter hentningen af `blocker.js` i forhold til `index.html` — samtidig, eller først bagefter?

5. Events og addEventListener()

JavaScript i browseren reagerer på brugerens handlinger gennem **events**. Hvert klik, tastetryk, scroll eller andre interaktioner sender et event, som vores kode kan “lytte” efter.

💡 **Event-drevet programmering** I browseren venter JavaScript på at ting sker, frem for at køre i et kontinuerligt loop:

Almindelige browser events:

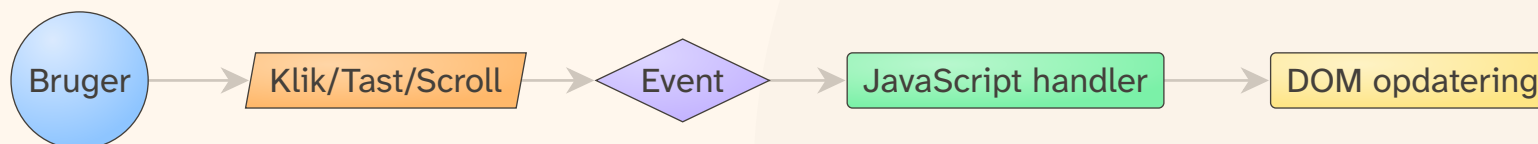
- **click** - Museklik på et element
- **submit** - Form sendes af sted
- **keydown/keyup** - Tastaturinteraktion
- **input** - Indhold i input felt ændres
- **resize** - Browservindue ændrer størrelse
- **load** - Siden er færdig med at load

addEventListener() syntax:

```
element.addEventListener('event', function)
```

Konkret eksempel:

```
let button = document.querySelector('button')
button.addEventListener('click', function() {
  console.log('Knappen blev klikket')
})
```



5.1 Event objekter og properties

Når et event udløses, modtager din event handler et **event objekt** med detaljerede oplysninger om hvad der skete:

💡 Event objektets vigtigste properties

```
button.addEventListener('click', function(event) {  
  console.log(event.target)  
  console.log(event.type)  
  console.log(event.clientX)  
  console.log(event.clientY)  
  event.preventDefault()  
})
```

Nyttige event properties:

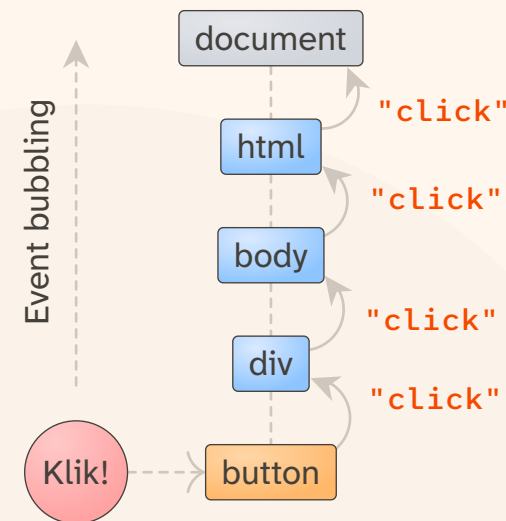
- **event.target** - Det element der blev påvirket
- **event.type** - Typen af event ('click', 'keydown', etc.)
- **event.clientX/Y** - Museposition relativt til viewport
- **event.key** - Hvilken tast blev trykket (ved keyboard events)
- **event.value** - Værdien i et input felt (ved input events)
- **event.preventDefault()** - Forhindrer standard browser handling (f.eks. form submission)

5.2 Event bubbling

Events “bobler” op gennem DOM træet fra det klickede element til dets forældre. Dette giver mulighed for event delegation og effektiv event handling på mange elementer ved at lytte på et fælles forældre element.

Eksempel på event bubbling:

```
<div>
  <button>Klik mig</button>
</div>
<script>
  document.querySelector('div').addEventListener('click',
    function(event) {
      console.log('Klik registreret på div')
      console.log('Event type:', event.type)
      console.log('Event target:', event.target)
    }
  )
</script>
```



6. Afprøv events

 Parvis  15 min

Eksperimentér med forskellige browser events og se hvordan de fungerer i praksis.

Sammen med din makker:

1. Keyboard events:

```
document.addEventListener('keydown',  
function(event) {  
  console.log('Tast trykket:', event.key)  
  if (event.key === 'Escape') {  
    console.log('Escape blev trykket!')  
  }  
})
```

Log hele **event** objektet og undersøg dets properties. Hvad kunne I forestille jer at bruge dem til?

2. Form events:

```
let input = document.querySelector('input')  
input.addEventListener('input', function(event) {  
  console.log('Value:', event.target.value)  
})  
let form = document.querySelector('form')  
form.addEventListener('submit', function(event) {  
  event.preventDefault() // Stop form submission  
  console.log('Form blev submittet')  
  console.log('Form data:', new FormData(form))  
})
```

Find selv på flere form og input events at eksperimentere med, såsom **change**, **focus**, og **blur**.

7. DOM manipulation: innerText og class

DOM (Document Object Model) er browserens representation af HTML-dokumentet som et træ af objekter.

Ved at manipulere DOM'en kan vi:

- **Ændre tekstindhold** dynamisk uden at genindlæse siden
- **Tilføje eller fjerne CSS klasser** for at ændre styling
- **Reagere på brugerinteraktioner** og opdatere interface i real-time
- **Skabe interaktive brugeroplevelser** der føles responsive og moderne

Tekstmanipulation:

```
let element = document.querySelector('h1')
element.innerText = 'Ny overskrift'
```

CSS klasse manipulation:

```
let button = document.querySelector('button')
button.classList.add('active')
button.classList.remove('disabled')
button.classList.toggle('highlighted')
```

7.1 innerText vs innerHTML vs textContent

I JavaScript er der flere måder at ændre elementers indhold, hver med forskellige anvendelsesområder og sikkerhedsimplikationer:

innerText:

- Kun ren tekst (ingen HTML tags)
- Respekterer CSS styling (skjulte elementer ignoreres)
- **Sikker** mod XSS-angreb
- Langsom ved store ændringer

```
element.innerText = 'Kun tekst <b>ingen fed</b>'  
// Resultat: "Kun tekst <b>ingen fed</b>"
```

innerHTML:

- Kan indeholde HTML tags
- Hurtig ved store ændringer
- **Farlig** hvis input ikke er valideret (XSS risiko)
- Trigger re-parsing af HTML

```
element.innerHTML = 'Med <b>fed tekst</b>'  
// Resultat: "Med fed tekst" (bold formateret)
```

textContent:

- Kun ren tekst (ingen HTML tags)
- Ignorerer CSS styling (alle text nodes inkluderet)
- Sikker mod XSS-angreb
- Hurtigere end innerText

```
element.textContent = 'Al tekst <span>også  
skjult</span>'  
// Resultat: "Al tekst også skjult"
```

💡 Tommelfingerregel

- **innerText**: Brugervendt tekst der skal være læsbar
- **innerHTML**: Når du har kontrol over HTML indholdet
- **textContent**: Når du vil have al tekst, uanset styling

Sikkerhed først: Brug aldrig **innerHTML** med bruger-input!

7.2 CSS class manipulation med classList

Grundlæggende operationer:

```
let element = document.querySelector('.button')
```

```
// Tilføj class
```

```
element.classList.add('active')
```

```
element.classList.add('highlighted', 'new')
```

```
// Fjern class
```

```
element.classList.remove('disabled')
```

```
// Toggle class (tilføj hvis ikke til stede, fjern
hvis til stede)
```

```
element.classList.toggle('selected')
```

```
// Check om class eksisterer
```

```
if (element.classList.contains('active')) {
  console.log('Element er aktivt')
}
```

Praktiske anvendelser:

→ Toggle navigation menu:

```
menu.classList.toggle('open')
```

→ Form validation feedback:

```
input.classList.add('error')
```

→ Loading states: `button.classList.add('loading')`

→ Theme switching:

```
document.body.classList.toggle('dark-theme')
```

→ Animation triggers: `element.classList.add('fade-in')`

CSS transition eksempel:

```
.button {
  transition: background-color 0.3s;
}

.button.active {
  background-color: #007bff;
}
```

8. Byg en character count i din chatbot

🕒 Solo ⌚ 45 min

🔗 Features:

- Vis antal tegn i real-time mens brugeren skriver
- Ændre farve når man nærmer sig en grænse (fx 280 tegn)
- Forhindre indsendelse hvis beskeden er for lang
- Giv visuel feedback for både normale og kritiske længder

💡 Udvidelser at eksperimentere med:

- **Animationer:** Smooth transitions mellem farveskift?
- **Progressbar:** Visuel indikator som fylder op?
- **Word count:** Vis både tegn og ord antal?
- **Disable submit:** Deaktiver send-knap når grænsen overskrides?

📄 Implementeringstips:

1. HTML kunne indeholde:

```
<div class="char-counter">  
  <span id="char-count">0</span>/280  
</div>
```

2. CSS styling:

```
.char-counter {  
  text-align: right;  
  font-size: 0.9em;  
  color: #666;  
}  
  
.char-counter.warning { color: orange; }  
.char-counter.danger { color: red; }
```

Prøv selv at skrive JavaScript logikken, der opdaterer tælleren og ændrer classes baseret på længde før du kigger på næste side...

8.1 JavaScript logik til character count

```
let input = document.getElementById('message-input')
let counter = document.getElementById('char-count')
let container = document.querySelector('.char-counter')

input.addEventListener('input', function(event) {
  let length = event.target.value.length
  counter.innerText = length

  // Fjern alle eksisterende klasser
  container.classList.remove('warning', 'danger')

  // Tilføj passende klasse baseret på længde
  if (length > 250) {
    container.classList.add('danger')
  } else if (length > 200) {
    container.classList.add('warning')
  }
})
```